Diffusion Models

From First Principles to LDAE

DDPM · DDIM · Latent Diffusion · Diffusion Autoencoders

reverse process - what diffusion models learn

Prepared by Avichal Trivedi · IIT Roorkee Internship

What is a generative model?

Two flavours of ML, and why generation is harder

Discriminative

Learns p(y/x) - given an image x, predict the label y.

Examples

- ResNet classifying cat vs. dog
- BERT classifying sentiment
- A pneumonia detector on X-rays

What it cannot do -

- Produce a new image of a cat
- Explain what a class "looks like"
- Imagine missing data

Generative

Learns p(x) - the distribution of images themselves.

Examples

- DALL-E, Midjourney, Stable Diffusion
- GPT generating new sentences
- GANs producing fake faces

Why it matters here -

- Sample new images from p(x)
- Imputation: fill in a missing scan
- Counterfactuals ("what if no AD?")

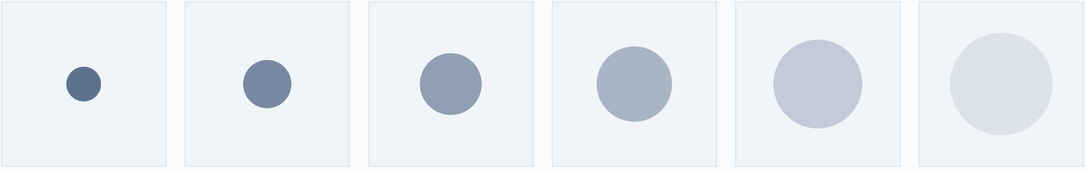
Diffusion is the current state-of-the-art way to learn p(x) for images.

The ink-in-water analogy

Why "diffusion" is exactly the right word

Forward process - physics

Add a drop of ink —> molecules scatter —> uniform haze



t=0 t=200 t=400 t=600 t=800 t=1000

In math, not chemistry

Replace molecules with pixel values.

Replace random bumping with adding Gaussian noise at each step.

After many steps, the image is statistically indistinguishable from pure random static - N(0, I).

Reversing physics is impossible.

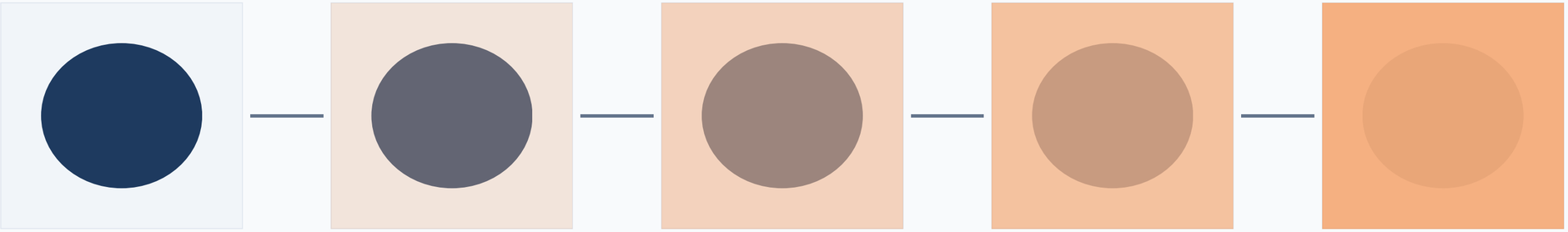
Reversing this maths is just a learning problem: train a network to undo one step of noise. Run that 1000 times —> fresh image.

Forward = trivial. Reverse = the entire job of the neural network.

The two processes

Forward (fixed, no learning) and Reverse (learned by a neural network)

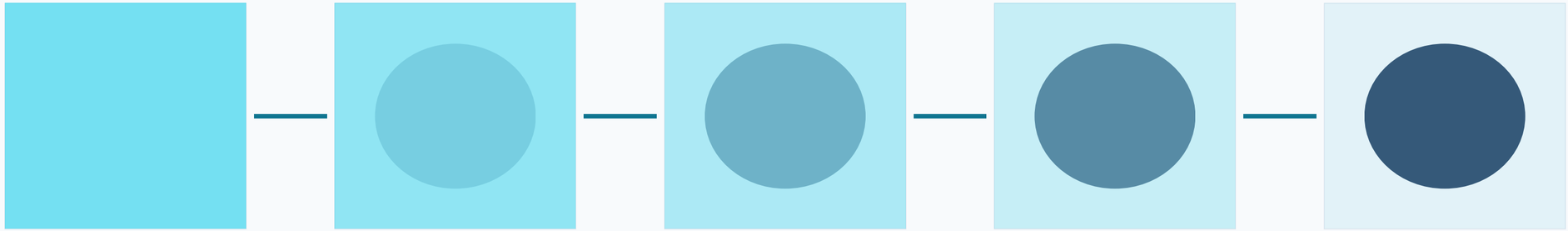
Forward q(x_t | x_{t-1}) - fixed schedule, no parameters



x_0 x_t x_t x_t x_T

clean image → add tiny Gaussian noise each step → pure noise

Reverse p_θ(x_{t-1} | x_t) - learned by a neural network



x_T x_t x_t x_t x_0

pure noise → network predicts noise & subtracts portion → clean image

The Math

Ho, Jain, Abbeel · NeurIPS 2020 · the paper that put diffusion on the map

1. Gaussian distribution

A bell curve described by mean μ and variance σ^2. The most common noise model in ML.

The normal/Gaussian: $N(x; \mu, \sigma^2 I)$.

Reparameterisation trick - sampling $x \sim N(\mu, \sigma^2 I)$ is the same as:

$$x = \mu + \sigma \cdot \varepsilon, \quad \varepsilon \sim N(0, I)$$

This lets us treat "noise" as a known quantity ε and a deterministic transformation. Used everywhere in DDPM.

2. Markov chain

A sequence $x_0 \to x_1 \to x_2 \to \ldots$ where each step depends only on the previous one.

Formally:

$$p(x_t \mid x_{t-1}, x_{t-2}, \ldots) = p(x_t \mid x_{t-1})$$

The forward diffusion process is exactly a Markov chain: each noisy version depends only on the previous noisy version.

Why it matters - Markov structure makes the joint factorise into a product of simple conditionals we can analyse.

3. KL divergence

A way to measure how different two probability distributions p and q are.

$$KL(p \| q) = \int p(x) \log[\, p(x) / q(x) \,]\, dx$$

KL = 0 when p and q are identical, and positive otherwise.

DDPM training is derived from minimising KL between the true reverse process and our learned reverse model — although the final loss simplifies dramatically (next slides).

DDPM - Forward process

What we do

Take a clean image x_0. At each step, mix it with a tiny bit of Gaussian noise. After T = 1000 steps, almost no signal remains.

The schedule β_t

β_t is a small positive number (typically 10^{-4} to 0.02). It controls how much noise is added at step t. The schedule {β_1, …, β_T} is fixed by hand - no learning.

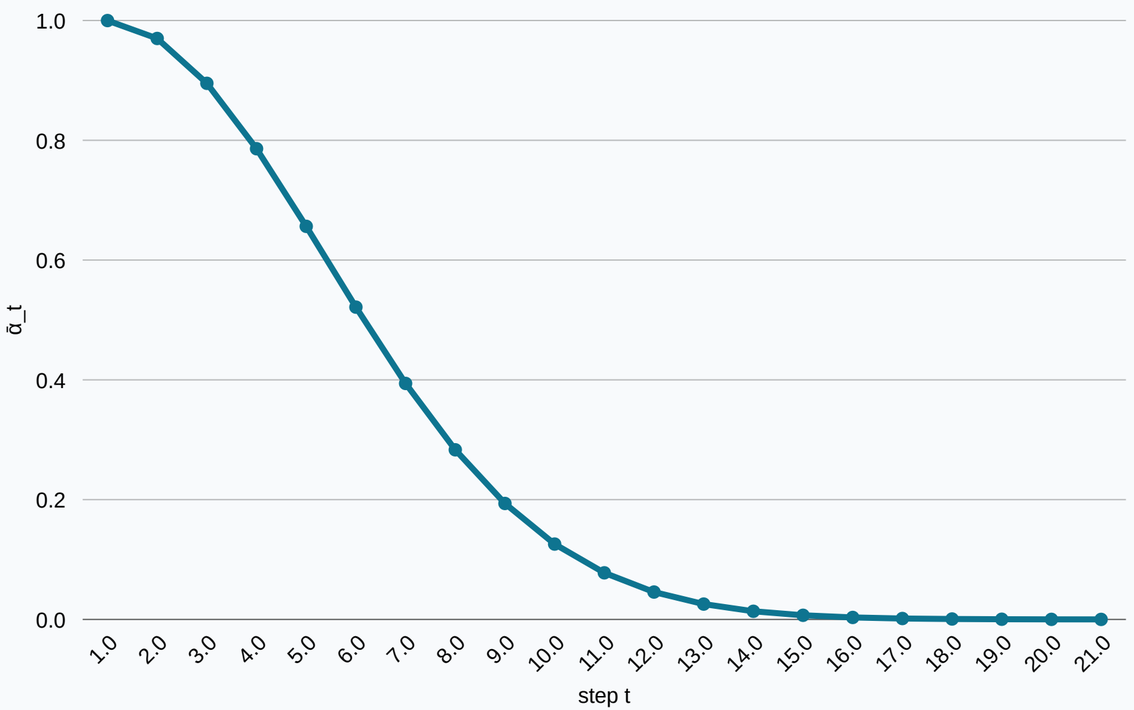
Why "sqrt(1 − β_t)" in the mean?

Without it, variance would grow unboundedly. The sqrt(1 − β_t) factor scales x_{t-1} down so that variance stays bounded as t —> T.

Single forward step

$$q(x_t \mid x_{t-1}) = N(x_t;\ \mathrm{sqrt}(1-\beta_t)\cdot x_{t-1},\ \beta_t \cdot I)$$

$$x_t = \mathrm{sqrt}(1-\beta_t)\cdot x_{t-1} + \mathrm{sqrt}(\beta_t)\cdot \varepsilon, \quad \varepsilon \sim N(0, I)$$



DDPM - Forward, in one shot

We never actually run 1000 steps during training - we sample x_t directly

Define cumulative product

$$\alpha_t = 1 - \beta_t$$
$$\bar{\alpha}_t = \alpha_1 \cdot \alpha_2 \cdot \ldots \cdot \alpha_t = \prod_{i=1}^{t} \alpha_i$$

Closed form (any t directly from x_0)

$$q(x_t \mid x_0) = N(\, x_t \,;\ \mathrm{sqrt}(\bar{\alpha}_t) \cdot x_0 \,,\ (1 - \bar{\alpha}_t) \cdot I \,)$$

$$x_t = \mathrm{sqrt}(\bar{\alpha}_t) \cdot x_0 + \mathrm{sqrt}(1 - \bar{\alpha}_t) \cdot \varepsilon$$

Intuition

ᾱ_t is the fraction of signal that survives.
(1 − ᾱ_t) is the fraction of pure noise.
At t = T, ᾱ_T ≈ 0, so x_T is ~ pure Gaussian noise.

Why this is gold for training

If we had to step from x_0 —> x_1 —> … —> x_t for every training example, training would be unbearably slow.

Because each step is Gaussian and we know the closed form for the cumulative effect, we can:

1. **Pick any random t between 1 and T.**
2. **Sample one ε from N(0, I).**
3. **Plug into the formula → get x_t in one operation.**

We now have a (clean image, noisy version, exact noise) triple to feed the network.

Reverse process - DDPM

If forward adds noise, reverse must remove it. But we don't know the true reverse.

What we wish we had vs. what we learn

True (intractable): $q(x_{t-1} \mid x_t)$

Approximation: $p_\theta(x_{t-1} \mid x_t) = N(x_{t-1};\ \mu_\theta(x_t, t),\ \sigma_t^2 I)$

Why approximate it as a Gaussian?

Sohl-Dickstein (2015) showed that if T is large enough and β_t is small enough, the true reverse step $q(x_{t-1} \mid x_t)$ is itself approximately Gaussian. So fitting a Gaussian (parameterised by a neural net) is principled - not a hack.

Step in reverse

ε_ϑ predicts the noise; subtracting (a portion of) it gives us x_{t-1}.

So instead of predicting μ_θ directly, the standard parameterisation has the network predict the noise ε that was added at step t. Mean and variance follow algebraically.

The training objective - DDPM

Predict the noise. That's the whole loss.

DDPM training loss (Ho et al. 2020, eq. 14)

$$L_{simple} = \mathbb{E}_{x_0, \varepsilon, t} \left[\| \varepsilon - \varepsilon_\theta(x_t, t) \|^2 \right]$$

Training algorithm (one step)

```
1.  Sample a clean image x_0 from the dataset.
2.  Sample t  ~  Uniform{1, …, T}.
3.  Sample ε  ~  N(0, I).
4.  Build x_t = sqrt(ᾱ_t)·x_0  +  sqrt(1 – ᾱ_t)·ε.
5.  Predict ε_θ(x_t, t) using the U-Net.
6.  Compute squared error  ‖ ε – ε_θ ‖².
7.  Backprop and update the U-Net weights.
```

Why this is remarkable

The full derivation comes from a variational lower bound (ELBO) over the joint distribution of all 1000 noisy versions.

It looks intimidating on paper. But after dropping a constant, simplifying KL terms between Gaussians, and reweighting, it collapses to:

“mean squared error between the noise you added and the noise the network predicted.”

That's why DDPMs are so much easier to train than GANs - there's no adversarial game, no mode collapse, just a regression problem.

DDPM Sampling - generating a new image

Run the trained noise predictor in reverse, 1000 times

DDPM reverse step

$$x_{t-1} = (1 / \mathrm{sqrt}(\alpha_t)) \cdot (x_t - ((1 - \alpha_t) / \mathrm{sqrt}(1 - \bar{\alpha}_t)) \cdot \varepsilon_\theta(x_t, t)) + \sigma_t \cdot z$$

$$\text{where } z \sim N(0, I) \text{ if } t > 1, \text{ else } z = 0$$

Sampling algorithm

```
1.  Sample x_T  ~  N(0, I).   (start with pure
noise)
2.  for t = T, T-1, …, 1:
       predict ε_θ(x_t, t)  with the U-Net
       compute x_{t-1}  using the equation above
3.  return x_0   ← the generated clean image
```

The cost

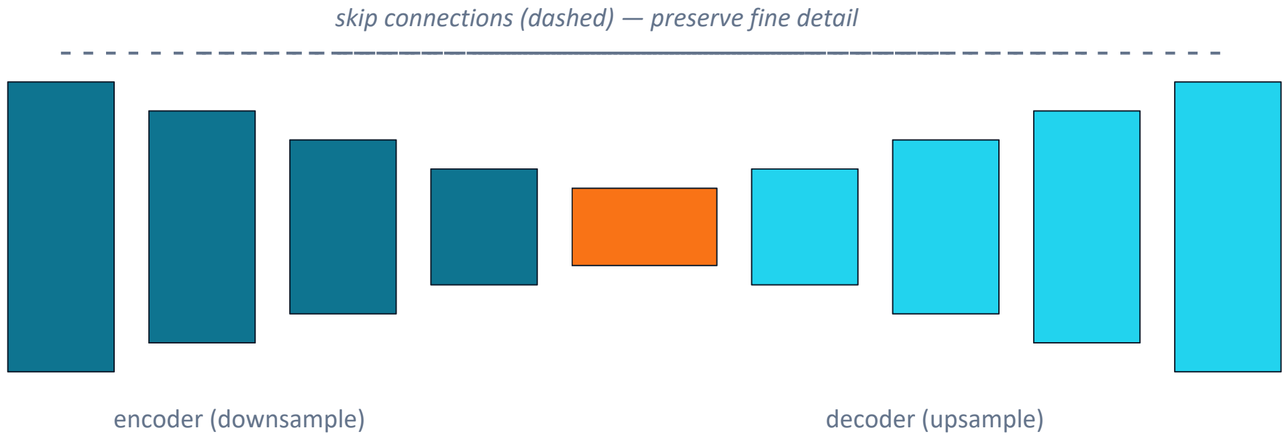
Each generation = 1000 U-Net forward passes (one per step).

On a 512×512 image: tens of seconds per sample on a strong GPU. Painful for production. Painful for research iteration.

Stochasticity:
Each step injects fresh noise z. Two runs from the same x_T → two different images. There is no way to encode a real image into a starting x_T.

What is ε_θ ? - the U-Net

The neural network that learns to predict noise



inputs: x_t , t **output: ε_θ(x_t, t)**

Original architecture from biomedical segmentation (Ronneberger 2015). The same shape: contract spatial dims while growing channels, then expand back. Skip connections fight the bottleneck's information loss.

Telling the U-Net which step it's on

The same network handles every t. How does it know?

- Sinusoidal positional embedding of t (same as Transformers).
- Pass through a small MLP —> time embedding vector.
- Inject into every residual block via AdaGN: scale & shift the normalised features.

Same mechanism is later used in DAE / LDAE to inject the semantic vector y_sem.

Two real problems with vanilla DDPM

Both will be fixed in the next chapters

Problem 1 - Slow

1000 reverse steps per image. Each step is a U-Net forward pass.

A 512×512 RGB image on a strong GPU: tens of seconds per sample. Generating a batch of training images for downstream use? Hours.

This kills practicality for real-time apps and makes research iteration expensive.

—> DDIM solves this. Same trained model, ~50 steps instead of 1000. We'll get there next.

Problem 2 - No way to encode a real image

DDPM is a pure generator. Given a real image x_0, there is no mechanism to recover the x_T that, when run through reverse sampling, would reproduce it.

Why? The reverse process is stochastic - different runs give different outputs even from the same x_T.

No encoding means: no editing real images, no representation learning, no interpolation between two real images.

—> DDIM's deterministic mode plus diffusion autoencoders solve this. Both come up shortly.

These two problems define the next 5 slides.

DDIM - fewer steps, same model

Song, Meng, Ermon · ICLR 2021 · the trick that made diffusion practical

The DDPM constraint

DDPM's forward process is Markov:

$$q(x_t \mid x_{t-1})$$

Each step depends only on the previous one. The reverse must run all 1000 steps to be principled.

DDIM's escape

It defines a non-Markov forward family that:

- preserves the same marginal $q(x_t \mid x_0)$,
- so any DDPM-trained network ε_θ also works here,
- but allows skipping steps cleanly.

DDIM reverse step

$$x_{t-1} = \mathrm{sqrt}(\bar{\alpha}_{t-1}) \cdot \hat{x}_0 + \mathrm{sqrt}(1 - \bar{\alpha}_{t-1} - \sigma_t^2) \cdot \varepsilon_\theta(x_t, t) + \sigma_t \cdot \varepsilon$$

$$\text{where } \hat{x}_0 = (x_t - \mathrm{sqrt}(1 - \bar{\alpha}_t) \cdot \varepsilon_\theta) / \mathrm{sqrt}(\bar{\alpha}_t)$$

The equation’s 3 pieces

1. Predicted clean x_0 (point we're heading to)
2. Direction pointing back toward x_t
3. Random noise scaled by σ_t (stochasticity)

DDIM- Two important points

σ controls stochasticity; striding controls speed

1 - stochasticity σ_t

$$\sigma_t = \eta \cdot \mathrm{sqrt}\left((1 - \bar{\alpha}_{t-1}) / (1 - \bar{\alpha}_t) \right) \cdot \mathrm{sqrt}(1 - \alpha_t)$$

η = 1 → stochastic — equivalent to DDPM

η = 0 → fully deterministic

When η = 0, the σ_t · ε term vanishes. Same starting noise x_T → same image, every time. The map x_T ↔ x_0 becomes a deterministic invertible function.

This is what unlocks DDIM inversion (next slide).

2 - step striding

Instead of stepping t = T, T−1, …, 1 (one step at a time), pick a sub-sequence:

$$\tau = (1000, 980, 960, \ldots, 20, 0)$$

Take 50 strides instead of 1000. Same trained ε_θ, same equation, just bigger jumps.

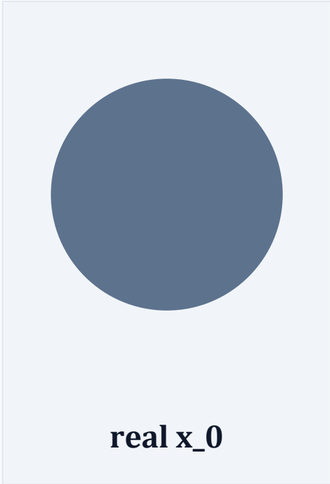
20× speedup at sampling time

Image quality holds remarkably well — the loss in fidelity is small for a 20× cost reduction.

Because DDIM uses the same trained network, you don't retrain anything - these are sampling-time choices.

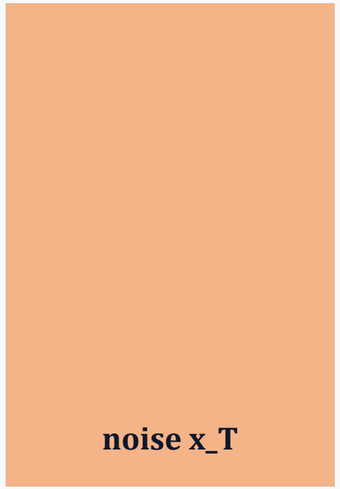
DDIM inversion - encoding real images

The deterministic map runs both ways.



DDIM inversion (encode): step the deterministic process forward t → t+1—>

<— DDIM sampling (decode): the standard reverse process with η=0



Forward (encoding) step - η = 0

$$x_{t+1} = \mathrm{sqrt}(\bar{\alpha}_{t+1}) \cdot \hat{x}_0(x_t) + \mathrm{sqrt}(1 - \bar{\alpha}_{t+1}) \cdot \varepsilon_\theta(x_t, t)$$

Why this is the foundation of LDAE

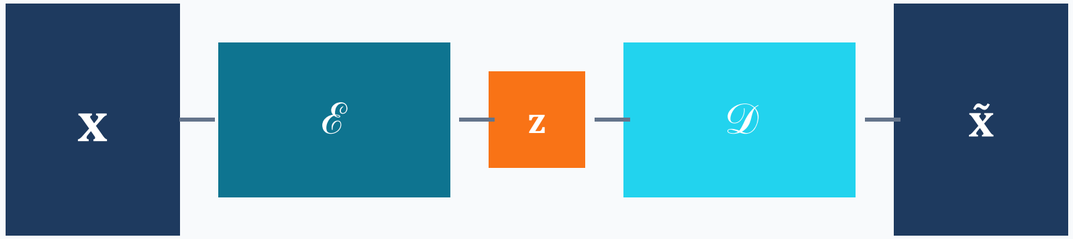
Without inversion: diffusion only generates from random noise.
With inversion: encode real image → edit in latent space → decode back.

LDAE uses this exact trick to capture subject-specific brain detail in x_T.

Reconstruction quality is excellent, but x_T captures low-level pixel detail - not semantics. We'll fix that with DAE.

Autoencoders & VAEs

Autoencoder (AE)



$$\text{Loss:} \quad L_AE = \| x - \mathcal{D}(\mathcal{E}(x)) \|^2$$

Train the encoder $\mathscr{E}$ and decoder $\mathscr{D}$ jointly to reconstruct x. Force z to be small → must learn a compact summary.

Variational Autoencoder (VAE)

Encoder outputs a distribution, not a single z:

$$q_\varphi(z \mid x) = N(z ; \mu_\varphi(x), \sigma_\varphi(x)^2 I)$$

Loss has two terms:

- Reconstruction error $\| x - \tilde{x} \|^2$
- KL divergence pulling $q_\phi(z|x)$ toward $N(0, I)$

Effect: latent space is smooth and Gaussian-like. Sampling $z \sim N(0, I)$ and decoding gives plausible images.

Pure VAE samples are blurry - but VAEs are excellent compressors of images into a clean latent space. We exploit this in LDM.

AE / VAE compress. Diffusion generates. Combining them = Latent Diffusion (next slide).

Latent Diffusion Models - Stable Diffusion

Rombach et al. CVPR 2022 · don't diffuse pixels, diffuse a compressed latent

1. Compress

Train (or reuse) an autoencoder.

512×512×3 = 786,432 numbers → 64×64×4 = 16,384 numbers.

48× fewer dimensions to diffuse over.

2. Diffuse latents

Train a DDPM on the latents z_0, not the images x_0.

Same DDPM math, much smaller tensor per step. U-Net only sees 64×64.

3. Decode at the end

Sample z_T ~ N(0, I), reverse-diffuse to z_0, then $\tilde{x} = \mathscr{D}(z_0)$.

Result: SD on a consumer GPU. Same trick LDAE uses for 3D MRI.

Reconstruction quality of LDM is bounded by the autoencoder. If $\mathscr{D}$ blurs detail, no diffusion magic recovers it. LDAE flags this.

Diffusion Autoencoders - intuition

Preechakul et al. CVPR 2022 · the key idea LDAE adapts

The question Can a diffusion model produce a meaningful semantic representation of an image - like CLIP, but generative?

DAE's answer: split the representation into two pieces

Semantic vector $y_{sem} \in \mathbb{R}^{512}$

Produced by a regular CNN encoder $Enc_\phi(x_0)$.

Encodes high-level content:

- person identity, age, expression
- for medical: AD vs. healthy, atrophy patterns

Compact, smooth, suitable for linear classifiers and interpolation.

Stochastic latent $x_T \in \mathbb{R}^{H \times W \times 3}$

Obtained via DDIM inversion of the conditional decoder.

Encodes low-level appearance:

- exact lighting, hair texture
- for medical: subject-specific anatomical noise

Same shape as the image - full pixel-level detail.

DAE - how the two pieces are trained

Standard DDPM loss, but the network is conditioned on y_sem

DAE training loss

$$L_DAE = \mathbb{E}_\{x_0, \varepsilon, t\} \left[\| \varepsilon - \varepsilon_\theta (x_t , t , y_sem) \|^2 \right]$$
$$\text{where} \quad y_sem = Enc_\varphi (x_0)$$

Why y_sem ends up semantic

ε_θ now sees both x_t and y_sem. To minimise loss, the network would love to ignore y_sem and just memorise the data - but the encoder is trained jointly.

The encoder is incentivised to put information in y_sem that helps the U-Net denoise. Useful info = global, semantic structure of x_0.

Low-level texture changes from noise step to noise step - useless to encode.

How y_sem enters the U-Net

Same way timestep t does: AdaGN (adaptive group norm)(to inject information while training diffusion models).

At every residual block, y_sem is projected to scale & shift parameters that modulate the normalised features.

This is the standard conditioning mechanism since Dhariwal & Nichol 2021. LDAE uses it identically.

What's left for x_T

After training, run DDIM inversion (conditional version: feed y_sem during inversion).

x_T captures the residual variability that y_sem couldn't explain — tiny anatomical asymmetries, exact intensity values, scanner-specific noise.

Reconstructing from (y_sem, x_T) gives near-pixel-perfect output.

PDAE - training cheaper with a frozen DDPM

Zhang, Zhao, Lin · NeurIPS 2022 · the version LDAE actually adapts

Frozen pretrained DDPM
(ε_θ)

🔒 *weights locked*
no gradient flows here

Semantic Encoder
Enc_ϕ (x_0) → y_sem

trainable
standard CNN backbone

Gradient estimator
G_ψ (x_t , y_sem , t)

trainable
shares encoder weights of frozen U-Net,
new decoder branch

PDAE objective - only φ and ψ are trained

$$L_PDAE = \mathbb{E}\left[\| \varepsilon - \varepsilon_\theta(x_t, t) + \lambda_t \cdot \sigma_t \cdot G_\psi(x_t, Enc_\varphi(x_0), t) \|^2 \right]$$

Heavy lifting (training a generative diffusion prior) is reused. Only the cheap parts - encoder + small decoder branch — train.

LDAE = LDM + PDAE for 3D brain MRI

Lozupone et al. 2026 · everything we just built, applied carefully

Three training stages, sequential

1 Compress
(LDM-style)

Train (or fine-tune) a perceptual VAE.

128×160×128 brain volume
→ 16×20×16 latent
170× compression.

Uses ℓ_1 + perceptual + KL + adversarial losses.

2 Pretrain DDPM
(in latent space)

Train an unconditional DDPM on the compressed latents z_0.

U-Net only operates on 16×20×16 - fast.

Result: a generative prior over the latent distribution p(z).

3 Add semantics
(PDAE-style)

Freeze stage 2.

Train semantic encoder (2.5D ConvNeXt + attention, ref. from paper) and gradient estimator.

Result: y_sem encodes AD, age, atrophy patterns. z_T encodes residuals.

Key results

AD diagnosis (linear probe on y_sem): AUROC 89.5%
Age prediction (linear probe on y_sem): MAE 4.16 years · Inference 20× faster than voxel-space DAE
Missing-scan interpolation: SSIM > 0.93 even at 24-month gaps

How the pieces stack

Each method solves a problem of the previous one

2020	**DDPM**	Formalised the noise-prediction loss. Slow, no encoding.
2021	**DDIM**	Non-Markov sampler. ~50 steps. η = 0 —> deterministic and invertible.
2022	**LDM (Stable Diffusion)**	Diffuse in compressed latent space, not pixel space. Practical at scale.
2022	**DAE**	Add a semantic encoder; (y_sem, x_T) split. Diffusion does representation learning.
2022	**PDAE**	Freeze the DDPM. Train only encoder + gradient estimator. Cheaper.
2026	**LDAE**	LDM + PDAE on 3D brain MRI. Linear-probe AD diagnosis at AUROC 89%. Missing-scan imputation.

Mental model · Diffusion = noise-prediction regression with a clever schedule. Everything else (DDIM, LDM, DAE, LDAE) is a plug-in that solves a specific limitation without retraining the core idea.

References

Papers and resources

Foundations

- **Lilian Weng - "What are Diffusion Models?"**
  Intro
- **Calvin Luo - "Understanding Diffusion Models: A Unified Perspective" (arXiv 2208.11970)**
  Derives DDPM, DDIM, score-matching cleanly from first principles. The reference.
- **Hugging Face Diffusers tutorial**
  Hands-on. Train a tiny DDPM on MNIST in ~50 lines.
- **Yang Song - score-based generative modeling blog**
  Alternative perspective on diffusion. Read after Lilian Weng.

I also referred to 3blue1brown, NormalizedNerd youtube channels for learning about concepts like KL divergence/Markov Chain.
Also to articles from Medium, SubStack, StackOverflow, etc.

Papers

- **Ho, Jain, Abbeel - DDPM (NeurIPS 2020)**
  The foundational paper.
- **Song, Meng, Ermon - DDIM (ICLR 2021)**
  Speed + determinism + inversion.
- **Rombach et al. - LDM / Stable Diffusion (CVPR 2022)**
  Diffuse in latent space.
- **Preechakul et al. - DAE (CVPR 2022)**
  Critical for LDAE.
- **Zhang, Zhao, Lin - PDAE (NeurIPS 2022)**
  The training scheme LDAE adapts.
- **Kazerouni et al. - Survey: Diffusion in Medical Imaging (2023)**
  LDAE with more medical background.